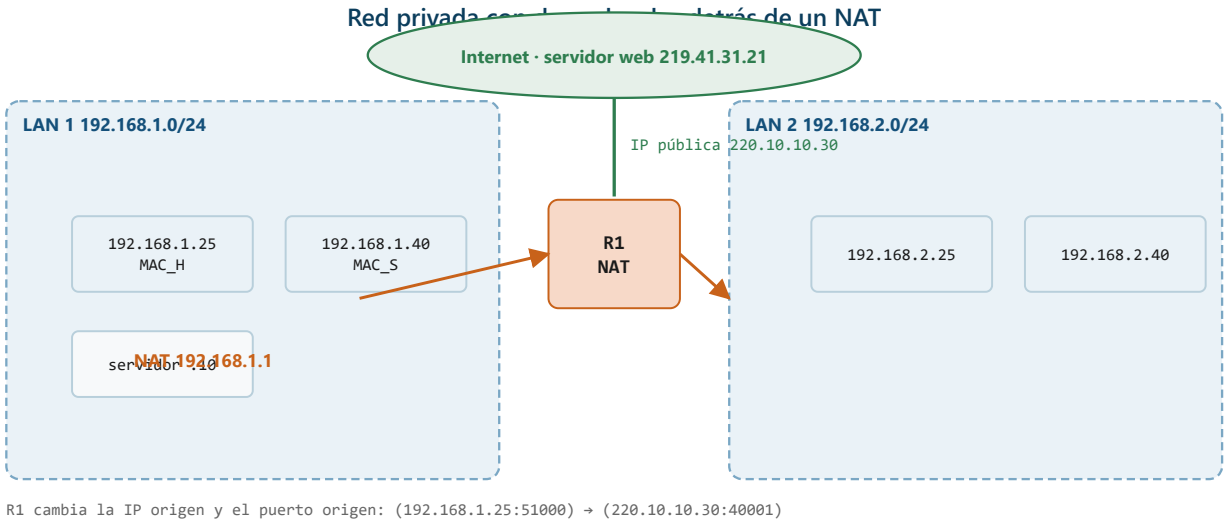
Clase 30 — Tablas de enrutamiento, next hop y ARP

Bloque: D (Capa de Red) · **Semana:** 10 · **Duración estimada:** 110 min **Correlativa:** Clase 29 (taller de subnetting) · **Sigue con:** Clase 31 (NAT, PAT y sus hacks) **Material de referencia:** Tanenbaum cap. 5 · RFC 826 (ARP) · RFC 1027 (proxy ARP) · RFC 4443 (ICMPv6, NDP) · RFC 8200 (IPv6)

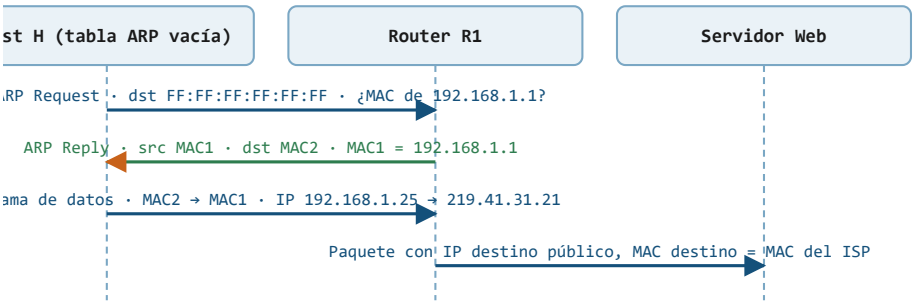


> 🔍 = RFC **no verificado** en `ACTUALIZACIONES.md` : no lo cites sin confirmarlo.

🎯 Al terminar esta clase vas a poder

1. Escribir la tabla de enrutamiento de un router y de un host en el **formato de tres columnas de la cátedra**, e interpretar cada columna.
2. Distinguir los **tres tipos de entradas**: conectada, aprendida y ruta por defecto.
3. Explicar el ***longest prefix match*** y por qué gana la **más específica**.
4. Ejecutar el **proceso de decisión completo** de un host y de un router.
5. Explicar por qué existe ARP y escribir el **formato del paquete ARP** campo por campo.
6. Describir **ARP Request (broadcast) → ARP Reply (unicast)** con MACs e IPs, y resolverlo enterito para un examen.
7. Describir la **tabla ARP** (aging, *gratuitous*, *spoofing*) y por qué **RARP** murió.

Secuencia ARP completa (el apartado b.i del Examen 2011-03)



> **Prerequisito del examen:** `2011-03 Problema 2 a, b.i y b.ii` · `2010-03 Pregunta 1a` · `2010-03 Problema 1 a.i-ii` · `2009-12 Problema 1 a-i-iii` · `2006-08 Pregunta 2` · `2001-02 Problema 1`.

PARTE 1 — TEORÍA

1.1 El formato de la tabla de enrutamiento

```
┌──────────────────────┬──────────────────────────┬──────────┐
│ Red destino / Máscara│ Gateway (next hop)       │ Distancia│
├──────────────────────┼──────────────────────────┼──────────┤
│ 192.168.1.0/24       │ C — Directamente conectado│    1     │
│ 0.0.0.0/0            │ 10.0.0.1                 │    3     │
└──────────────────────┴──────────────────────────┴──────────┘
```

Columna	Qué es	Ojo
Red destino / Máscara	El prefijo que dice "todo lo que cae acá, va por acá". **La máscara es parte de la clave**: `192.168.1.0/24` y `192.168.1.0/25` son entradas **distintas**.	Sin máscara no se puede hacer *longest prefix match*
Gateway (next hop)	La IP del **router vecino** por el que se sale, o **C** (*connected*) si el destino es una red **directamente conectada**.	Es un **vecino**, no el destino final: el paquete va al `10.0.0.1` y ese router decide después
Distancia (o **adquisición**)	Cuántos saltos cuesta, o de dónde salió la ruta.	Una red conectada tiene distancia **1**, no 0: el "0 saltos" sos vos

Sobre el `C`. *"El prefijo de la columna 1 es una de mis redes de interfaz, no hace falta ningún router para llegar"*. No la aprendió nadie: la puse yo, y es la **única entrada que nunca se borra** mientras la interfaz esté arriba. La notación equivalente del `2008-07 P1b` es una columna **Adquisición** con `C` (conectada), `R` (por RIP) y `O` (*other*, la que anuncia el ISP).

⚠️ `C` **también** puede llevar gateway: es el caso de un router con **dos interfaces en la misma red**, o de una **red de tránsito**. `10.0.3.0/24 → C → 10.0.3.1` quiere decir: *"esa red es mía, la tengo en `eth0`, y mi dirección therein es 10.0.3.1"*.

1.2 Los tres tipos de entradas

```
① CONECTADA                  ② APRENDIDA                  ③ POR DEFECTO
─────────────                ─────────────                ─────────────
Red de una de mis interfaces. Red que NO es mía.          Red destino = 0.0.0.0/0
Gateway = C.                 Gateway = la IP de un        ("cualquiera").
Distancia = 1.               vecino.                      Gateway = el router
Nunca se borra.               Distancia = n° de saltos.    de salida.
La pongo YO.                 La aprendo de otro router.   La más cara que conozco.

192.168.1.0/24 → C  (1)      192.168.0.0/24 → 10.0.0.2 (2)  0.0.0.0/0 → 10.0.0.1 (3)
```

Por qué la ruta por defecto tiene distancia propia. Es una ruta más: si hay un camino específico, gana (§1.3); si no, usás el default. Y si **tampoco** hay default, el router **descarta el paquete y manda un ICMP**

tipo 3 código 0 (*net unreachable*). **La tabla de un host** es la versión mínima: una red local (`C`) + la default; con **dos** interfaces, dos redes locales + una default (Ejercicio 2.1).

1.3 *Longest prefix match*

El router no elige "la ruta que más se parece", sino **la de prefijo más largo** que todavía coincida con el destino.

```
Tabla de un router
  192.168.1.0/24      →  eth0          (conectada,           distancia 1)
  192.168.1.128/25    →  10.0.0.2      (aprendida de vecino, distancia 2)
  0.0.0.0/0           →  10.0.0.1      (por defecto,         distancia 3)

Llega un paquete con IP destino 192.168.1.200

  ¿192.168.1.0/24  coincide?   192.168.1.200 & 255.255.255.0   = 192.168.1.0    SÍ
  ¿192.168.1.128/25 coincide?  192.168.1.200 & 255.255.255.128 = 192.168.1.128 SÍ
  ¿0.0.0.0/0        coincide?   siempre SÍ

  El prefijo MÁS LARGO que coincide es /25  (25 > 24 > 0)  ⇒  sale por 10.0.0.2
  Y la entrada /24 NO sobra: sirve para 192.168.1.1 … .127, que no caen en el /25.
```

El test de coincidencia, el mismo del subnetting: `D AND máscara == red`.

Por qué el más específico y no el de menor distancia. La `/25` es más cara (2 saltos) pero trata un **subconjunto** de la `/24`, así que **debe** ganar. Si ganara la de menor distancia, la `/24` no se podría usar nunca y las subredes sumarizadas no existirían.

⚠️ `192.168.1.0/25` y `192.168.1.128/25` **no** "se parten": para el router son dos redes, y cada host tiene una tabla distinta.

1.4 El proceso de decisión completo

En un host (el 90 % de los casos que se preguntan):

```
H llega un paquete con IP destino D
          |
          ▼
┌──────────────────────────────────────────┐
| ¿D está en la tabla con más prefijo que  |  NO ──→ usar la entrada 0.0.0.0/0
| coincida?  (longest prefix match sobre   |         → el default gateway
| TODA la tabla, no solo la primera)       |          SIN default: se descarta
└──────────────────────────────────────────┘
               | SÍ
               ▼
┌──────────────────────────────────────────┐
| ¿La entrada dice "C" (conectada)?        |  NO ──→ el paquete va al router vecino
└──────────────────────────────────────────┘          (se resuelve el MAC con ARP)
               | SÍ
               ▼
Entregar el paquete a la capa 4 sin tocar nada más.
```

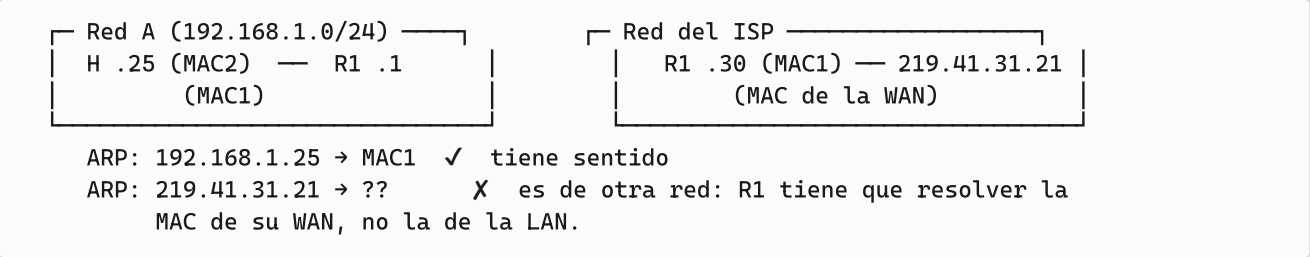
En un router son los mismos dos pasos, más un caso propio: si el destino es una dirección de **broadcast o multicast de una de sus propias subredes**, tiene que generar el paquete él mismo (es el paso 3 del `traceroute` de la `2006-08 P1`).

Lo que NO hace el router, y es el error conceptual más común: **no** mira la IP de origen para decidir, y **no** reescribe las IP al reenviar — solo cambia las tramas de capa 2 **de su enlace de salida**. Por eso la IP **no cambia de salto** (salvo NAT, Clase 31) y la **MAC cambia en cada uno**.

1.5 ARP: por qué existe

El problema, en una frase. En IPv4 la dirección es **jerárquica y lógica**: `192.168.1.25` dice "esta IP está en la red 192.168.1.0/24" y nada del hardware. En Ethernet la dirección es **plana y física**: una MAC de 48 bits que **no codifica nada** sobre en qué red estás. **Son dos espacios de nombres distintos para identificar lo mismo.**

Por qué no alcanza con la IP. La trama de Ethernet tiene un campo obligatorio de **MAC destino**. El host no puede poner ahí una IP, ni inventar una MAC: tiene que **preguntar** "*¿qué MAC tiene la IP 10.0.0.1?*". Y hay que preguntárselo a alguien, porque **la MAC solo tiene sentido local** dentro de un enlace: por eso **ARP nunca cruza un router** — cada router responde solo por sus propias interfaces.



> **ARP = Address Resolution Protocol**, protocolo auxiliar de la suite TCP/IP que resuelve la correspondencia IP ↔ MAC **en un mismo enlace**. Definido en la `RFC 826`.

1.6 El formato del paquete ARP

Un paquete ARP viaja **dentro** de una trama Ethernet, con `Ethertype = 0x0806` (que es lo que le dice al receptor "esto no es IP, es ARP").

```
TRAMA ETHERNET (14 bytes)
┌────────────┬────────────┬──────────────┬──────────────────────────────────┐
│ MAC dest 6 │ MAC orig 6 │ Tipo  2      │  payload ARP (28 bytes)          │
│            │            │  0x0806      │                                  │
└────────────┴────────────┴──────────────┴──────────────────────────────────┘

PAYLOAD ARP (28 bytes)
┌────────────┬────────────┬──────────┬──────────┬──────────┬────────────────┬────────────┐
│ HW type 2  │ Proto t.2  │ HWl 1    │ Prol 1   │Opcode 2  │ Sender HW  6   │ Sender IP4 │
│  =1 Ether  │ =0x0800    │  = 6     │  = 4     │1=req     │ = MAC orig     │ = IP orig  │
│            │ (IPv4)     │          │          │2=reply   │                │            │
├────────────┼────────────┼──────────┴──────────┴──────────┴────────────────┴────────────┤
│ Target IP4 │ Target HW6 │  Total: 14 + 28 = 42 bytes                                 │
│= IP que    │=00:00:00:00:00:00   en el REQUEST: se desconoce                        │
│  se busca  │  (se desconoce)   en el REPLY: la MAC de quien preguntó                 │
└────────────┴────────────┴─────────────────────────────────────────────────────────────┘
```

Los dos campos que hacen toda la magia:

Campo	En un **Request**	En un **Reply**	Por qué
Target IP address	La IP **buscada**	La IP **del que preguntó**	El request necesita saber a quién preguntar; el reply, a quién contestar
Target HW address	`00:00:00:00:00:00`	La MAC **del que preguntó**	El request todavía no la sabe

Esa simetría (el request pregunta con `Target IP`, el reply responde con `Sender IP`) es lo que hace que todo funcione sin una segunda ronda de pregunta.

1.7 La secuencia ARP completa

```
H  (192.168.1.25, MAC 2)                R1 (192.168.1.1 / 220.10.10.30, MAC 1)
 │                                                          │
 │ 1. ARP REQUEST                              EtherType 0x0806│
 │    MAC destino : FF:FF:FF:FF:FF:FF  (broadcast)          │
 │    MAC origen  : MAC 2                                   │
 │    Opcode 1 · Sender MAC MAC2 · Sender IP 192.168.1.25   │
 │    Target HW 00:00:00:00:00:00 · Target IP 192.168.1.1   │
 ├──────────────────────────────────────────────────────────►│
 │                                                          │
 │ 2. ARP REPLY                                EtherType 0x0806│
 │    MAC destino : MAC 2   ← unicast a quien preguntó      │
 │    MAC origen  : MAC 1                                   │
 │    Opcode 2 · Sender MAC MAC1 · Sender IP 192.168.1.1    │
 │    Target HW MAC2          · Target IP 192.168.1.25      │
 │◄──────────────────────────────────────────────────────────┤
 │                                                          │
 │ 3. H guarda  192.168.1.1 → MAC 1  en su TABLA ARP        │
 │ 4. H arma la trama: MAC destino MAC1 · MAC origen MAC2   │
 │    y dentro, el paquete IP  192.168.1.25 → 219.41.31.21  │
 ├──────────────────────────────────────────────────────────►│
```

⚠️ **Request broadcast, reply unicast.** Es lo que dicen la `RFC 826` y las soluciones oficiales del `2010-03 P1a` y del `2006-08 P2`.

⚠️ **Las dos MAC del mismo sentido.** En el reply, **MAC origen** = la de quien responde (MAC 1, la de R1) y **MAC destino** = la de quien preguntó (MAC 2). Confundirlas es el error más común del apartado.

⚠️ **La tabla ARP se llena con lo que dice el campo *Sender*, no con lo que se preguntó.** H preguntó por `192.168.1.1` y guarda `192.168.1.1 → MAC 1`, que es justo el `Sender IP` del reply.

1.8 La tabla ARP: *aging*, *gratuitous* y *spoofing*

```
┌──────────────────┬──────────────────┬──────────────┬──────────────┐
│ IP               │ MAC              │ Interfaz     │ Antigüedad   │
├──────────────────┼──────────────────┼──────────────┼──────────────┤
│ 192.168.1.1      │ MAC 1            │ eth0         │  0:00:12     │
│ 192.168.1.77     │ MAC 5            │ eth0         │  1:42:03     │
│ 220.10.10.29     │ MAC WAN          │ eth1         │  3:07:55     │
└──────────────────┴──────────────────┴──────────────┴──────────────┘
```

- **Qué guarda:** la correspondencia **IP → MAC**, más la **interfaz** por la que se alcanzó (necesaria en routers multihoming) y un **temporizador de antigüedad**.
- **¿Por qué una tabla?** Por **retardo** (un ARP son 2 tramas y 1 RTT) y por **carga** (un broadcast por paquete satura la LAN). Es un caché, no corrección.
- **Aging:** cada entrada tiene un temporizador que **se renueva** al usarse y **expira** si no se usa. Los valores son **de implementación**, no los manda IEEE: del orden de **30 s a 5 minutos** en Windows y Linux. Expirar es **necesario**, porque un host puede cambiar de MAC y la entrada vieja lo mandaría al lugar equivocado.
- **ARP *gratuitous*:** un Request donde **Sender IP = Target IP**, o sea *"yo soy el dueño de esta IP"*, a broadcast y **sin respuesta**. Sirve para **anunciar** una dirección nueva al arrancar, **detectar duplicados**, **actualizar las cachés** tras un *failover* de gateway, y como **anuncio de VRRP**.
- ⚠️ **Y es la fuente clásica de *ARP spoofing: si un atacante manda un gratuitous diciendo "192.168.1.1 es MI MAC", todos los hosts de la LAN le creen* y le mandan su tráfico. Eso es*** cache poisoning***, la base de casi todos los ataques* man-in-the-middle* de LAN. Las defensas:** ARP inspection **en los switches y** fijar la MAC del gateway** en cada host.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - `RFC 826`: request en **broadcast**, reply en **unicast**, opcodes 1 y 2; los 14 bytes de Ethernet + 28 de payload, con los cuatro campos de direcciones. - La tabla ARP es un **caché con expiración**. - El *gratuitous* se sigue usando (arranque, DHCP, failover, VRRP) y **sigue siendo una fuente clásica de *ARP spoofing***. - La convención "C" y el formato de tres columnas.

Cambió o hay que reencuadrar: - **En IPv6 no hay ARP: hay Neighbor Discovery** (`RFC 4443`), que hace lo mismo con mensajes **NS/NA de ICMPv6** sobre **multicast**, y además agrega **autoconfiguración** y **DAD**. No hay *proxy ARP* en IPv6. Es el contraste que pide el `2010-12 P5`. - **`Proxy ARP` (`RFC 1027`) sigue existiendo** en VPNs sitio a sitio, pero es cada vez más un **anti-patrón** de la era IPv4-NAT. - **El *aging* es de implementación**: nunca lo presentes como "el estándar dice X".

> **Consecuencia para el examen: nada cambia** en el formato ni en la aritmética. Lo único a añadir, si te piden actualizar, es el NDP como equivalente en IPv6.

1.9 RARP: el problema inverso, y por qué murió

```
ARP   "¿qué MAC tiene la IP 192.168.1.25?"     →  respuesta:  la MAC
RARP  "¿qué IP y qué MAC tiene esta tarjeta?"  →  respuesta:  la IP + la MAC
```

El problema que resuelví. Un host recién arrancado, con la tarjeta Ethernet en la LAN pero **sin dirección IP**, no puede usar DHCP: no tiene con quién hablar. La pregunta de RARP es esa: "*sos una interfaz con esta MAC; decime qué IP me corresponde*". El servidor RARP tiene una tabla **MAC → IP** y contesta. Se define en la `RFC 903`, y sus opcodes (3 y 4) viven en el formato de la `RFC 826`.

Por qué quedó obsoleto: 1. **Solo devolvía la IP.** No entregaba **máscara**, ni **gateway por defecto**, ni **DNS**. Con una IP suelta no se puede enrutar nada: el host no sabe si el destino es local o remoto. 2. **El servidor necesitaba una entrada por host**, escrita a mano: agregar un host implicaba editar la tabla del servidor. 3. **BOOTP** lo reemplazó (🔍 `RFC 951`): pasó a entregar IP, máscara, gateway y archivo de arranque, todo en un mensaje, y de forma automática desde un *pool*. 4. **Después llegó DHCP** (🔍 `RFC 2131` / `2132`), que es BOOTP renovado con *leases* (alquilaciones con vencimiento), lo que resolvió el problema de la dirección fija atada al hardware.

> ⚠️ **tema en desuso.** Si un examen te pregunta RARP, la respuesta es "se usaba en los 80, quedó obsoleto con BOOTP y DHCP". Si aparece como **pregunta teórica**, es porque quieren ver si entendés el salto de "dirección fija" a "dirección alquilada con vencimiento".

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La tabla de un host con **dos** interfaces

> **Fuente:** Examen de marzo de 2001, `sirc0102`, Problema 1 (30 pts). *Enunciado: "Se dispone de un computador que se comunica con otros mediante TCP/IP. Para hacerlo posee dos tarjetas ethernet que están conectadas a diferentes LAN's. El sistema conoce: 1) la dirección IP, la máscara de subred y la dirección IP del gateway por defecto (si ha sido especificado) para cada una de las dos tarjetas. 2) La tabla de ruteo estática, con entradas del tipo `<dirección de red de destino, máscara de subred, gateway a utilizar>`. Se pide: escribir el proceso que determina por cuál o cuáles tarjetas se debe enviar un paquete IP dada la dirección IP destino."*

La lógica, en cuatro pasos, en este orden exacto:

```
Paso 1 — ¿el destino es LOCAL de alguna de las dos LAN?
    for i en {1,2}:   si  D & mascara[i] == ip[i] & mascara[i]  ⇒  tarjeta i

Paso 2 — ¿hay entrada en la tabla estática que coincida?
    for ℓ:              si  D & mascara[ℓ] == red[ℓ] & mascara[ℓ]  ⇒  manda ℓ
    (si hay varias, gana la de prefijo MÁS LARGO)

Paso 3 — ¿por dónde sale el gateway de esa línea?
    si  gw[ℓ] & mascara[1] == ip[1] & mascara[1]  ⇒  tarjeta 1
    si  gw[ℓ] & mascara[2] == ip[2] & mascara[2]  ⇒  tarjeta 2

Paso 4 — si no hubo coincidencia, los gateways por defecto:
    ¿existen los DOS?  ⇒  se manda por AMBAS tarjetas       (devuelve 3)
    ¿existe uno solo?  ⇒  por esa                           (1 o 2)
    ¿ninguno?           ⇒  se descarta                      (0)
```

```
// Tarjeta_red[i] = { dir_IP, Sn_mask, Def_gw }   i en 1..2
// Tabla_ruteo[l] = { dir_IP, Sn_mask, Def_gw }
// Devuelve: 0 ninguna · 1 tarjeta 1 · 2 tarjeta 2 · 3 AMBAS

function selec_tarjeta (dirIP: IP_addr): int
begin
  // Paso 1: ¿es una de mis LANs?   (AND bit a bit)
  if (dirIP and Tarjeta_red[1].Sn_mask =
      Tarjeta_red[1].dir_IP and Tarjeta_red[1].Sn_mask) then return 1;
  if (dirIP and Tarjeta_red[2].Sn_mask =
      Tarjeta_red[2].dir_IP and Tarjeta_red[2].Sn_mask) then return 2;

  // Paso 2: tabla de ruteo estática
  res := 0;
  for lin := 1 to MAX_ROUTE do
    if (dirIP and Tabla_ruteo[lin].Sn_mask =
        Tabla_ruteo[lin].dir_IP and Tabla_ruteo[lin].Sn_mask) then
      res := lin;            // ⚠ la oficial se queda con la ÚLTIMA, no con
                             //   la más específica (ver análisis)
  if (res > 0) then begin
    // Paso 3: ¿el gateway está en la LAN de la tarjeta 1 o de la 2?
    if (Tabla_ruteo[res].Def_gw and Tarjeta_red[1].Sn_mask =
        Tarjeta_red[1].dir_IP and Tarjeta_red[1].Sn_mask) then return 1;
    if (Tabla_ruteo[res].Def_gw and Tarjeta_red[2].Sn_mask =
        Tarjeta_red[2].dir_IP and Tarjeta_red[2].Sn_mask) then return 2;
  end;

  // Paso 4: gateways por defecto
  if (existe Tarjeta_red[1].Def_gw) and (existe Tarjeta_red[2].Def_gw) then
    return 3;                              // por AMBAS
  if (existe Tarjeta_red[1].Def_gw) then return 1;
  if (existe Tarjeta_red[2].Def_gw) then return 2;
  return 0;                                // no hay por dónde salir
end;
```

Después del `selec_tarjeta`, dos cosas más (acá está el punto de unión con esta clase): 1. **La IP de origen es la de la tarjeta por la que se sale**: `tabla_ruteo[n].dirIP`. 2. **El ARP es del destino final, no del destino del paquete**: si el destino es **local** → ARP de `dirIP`; si salís por la tabla o por un default → **ARP del gateway**. Es el criterio de §1.4: primero se decide la **interfaz de salida**, y solo después se resuelve la MAC de **a quién** se entrega la trama.

> **Análisis de la solución oficial:** la lógica y el orden son correctos (LAN → tabla estática → default). Dos cosas a corregir: (a) el `res := lin` dentro del `for` se queda con la **última** coincidencia, no con la de **prefijo más largo**; con dos entradas que coinciden devuelve la que vino último, que puede no ser la más específica. Hay que comparar las máscaras y quedarse con la **más larga**. (b) Conviene recordar que **los dos AND tienen que hacerse bit a bit**, no con comparación de strings.

2.2 Ejercicio 2 — ARP entre dos LANs, con un router en el medio

Enunciado. H1 quiere alcanzar un servidor en Internet, a través de un router, y su tabla ARP está **vacía**.

```
 H1 (192.168.1.10, MAC1)     Router (LAN 192.168.1.1, MAC2 · WAN 203.0.113.5, MAC3)

1) H1 necesita la MAC de 192.168.1.1  (el destino NO es local: es remoto)
   ── ARP REQUEST  MAC dest ff:ff:ff:ff:ff:ff · MAC orig MAC1 ──────────►
      Sender IP 192.168.1.10 · Target IP 192.168.1.1
   ◄── ARP REPLY    MAC dest MAC1 · MAC orig MAC2 ──────────────────────
      Sender IP 192.168.1.1 · Target IP 192.168.1.10

2) H1 arma la trama de datos, SIN tocar el paquete IP:
   MAC dest MAC2 · MAC orig MAC1 · Ethertype 0x0800
   IP origen 192.168.1.10  →  IP destino (la del servidor, digamos 142.250.1.1)
   ──────────────────────────────────────────────────────────────────►

3) El router tira la MAC2 y el Ethertype, lee la IP destino, ve que no es de
   ninguna de sus redes, consulta SU tabla (0.0.0.0/0 → 203.0.113.1), hace ARP
   por 203.0.113.1 en su WAN y reenvía con MAC destino = la del ISP.
```

⚠️ **Lo que enseña, en tres líneas:** la **resolución de MAC es local a cada enlace**; el **paquete IP viaja intacto** de extremo a extremo (aquí; NAT lo rompe, Clase 31); y el router decide la **ruta** con la IP destino y la **trama** con el siguiente salto.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen `2011-03`, Problema 2 a, b.i y b.ii — tablas, ARP y MACs

> **Fuente:** Examen `eirc1103` / `sirc1103`, 21 de febrero de 2011, Problema 2 (30 puntos), apartados a, b.i y b.ii. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

Enunciado (literal)

> Una empresa tiene una red A (LAN Ethernet con direccionamiento privado) con 15 servidores y 70 puestos de trabajo, y se desea que puedan acceder a un servidor web en Internet con dirección IP `219.41.31.21`. Para ello el router R1 implementa NAT utilizando su dirección IP pública. **a)** Especifique la tabla de enrutamiento de R1 y del Host H. **b)** i. Suponiendo que la tabla ARP del Host H está vacía, indicar los mensajes ARP (MAC de origen, MAC de destino y descripción del contenido) intercambiados en la Red A cuando el Host H inicia su comunicación con Servidor Web (asuma que Host H conoce la dirección IP de Servidor Web). ii. Indique las direcciones MAC de las tramas y las direcciones IP de los paquetes correspondientes a un flujo de datos entre el Host H y el Servidor Web considerando ambos sentidos (considere únicamente la Red A).

Datos de la figura: H = `192.168.1.25`, **MAC 2**. R1 = `192.168.1.1` (**MAC 1**) en la LAN, y `220.10.10.30` en el enlace `220.10.10.28/30`.

a) Las dos tablas

```
Tabla del host H                          Tabla de R1
Red destino / Máscara   Gateway          Red destino / Máscara    Gateway
──────────────────────────────────────   ──────────────────────────────────────
192.168.1.0/24          C                192.168.1.0/24           C     (1)
0.0.0.0/0               192.168.1.1 (2)  220.10.10.28/30          C     (1)
                                           0.0.0.0/0                220.10.10.29 (2)
```

⚠️ **Las dos entradas `C` de R1, y por qué.** R1 tiene **dos** interfaces: una en la red A y otra en el enlace WAN, y cada una genera su entrada conectada. Es el error más común poner una sola: si a R1 le llega un paquete para `220.10.10.29` (el router del ISP) y no tiene la entrada `/30`, lo tira por la interfaz equivocada o lo descarta.

b.i) Los dos mensajes ARP

	MAC orig	MAC destino	Tipo y contenido
Request (H)	MAC 2	FF:FF:FF:FF:FF:FF (broadcast)	ARP Request preguntando por la MAC asociada a la IP 192.168.1.1 · Opcode 1 · Sender MAC MAC2 · Sender IP 192.168.1.25 · Target MAC 00:00:00:00:00:00 · Target IP 192.168.1.1
Reply (R1)	MAC 1	MAC 2 (unicast al que preguntó)	ARP Reply respondiendo que la MAC asociada a la IP 192.168.1.1 es MAC 1 · Opcode 2 · Sender MAC MAC1 · Sender IP 192.168.1.1 · Target MAC MAC2 · Target IP 192.168.1.25

> **Por qué un broadcast y no un unicast:** H **no sabe** la MAC de `192.168.1.1`, así que no tiene a quién dirigir la pregunta. El reply sí puede ser unicast, porque **ya conoce** la MAC del que preguntó (la tiene en el `Sender MAC` del request que recibió).

b.ii) MACs e IPs de las tramas, en los dos sentidos

```
Sentido H → Servidor Web                Sentido Servidor Web → H
MAC origen  : MAC 2                     MAC origen  : MAC 1
MAC destino : MAC 1                     MAC destino : MAC 2
IP  origen  : 192.168.1.25              IP  origen  : 219.41.31.21
IP  destino : 219.41.31.21              IP  destino : 192.168.1.25
```

> **El análisis que hay que hacer en voz alta** (es lo que suma puntos): "*mirá las cuatro IP: en los dos sentidos son las mismas. Lo único que cambia entre un paquete y su respuesta es el orden: se invierten origen y destino. Las MAC también se invierten, pero no porque el paquete 'vuelva': porque la respuesta se emite desde la interfaz de R1 hacia H, y esa trama tiene que llevar la MAC de H como destino. Las IP no se tocan porque este es el tramo de la capa 3 dentro de la red A, antes de que R1 aplique el NAT — que es exactamente lo que pide el apartado c.*"

⚠️ **El error clásico:** poner `IP destino 220.10.10.30` en el sentido Web → H. **No.** `220.10.10.30` es la IP que R1 *muestra hacia afuera* (NAT), y por el punto de vista de H el destino sigue siendo él mismo. El NAT se explica entero en el apartado c, que es la Clase 31.

3.2 Examen `2010-03`, Pregunta 1 a — ARP entre dos hosts

> **Fuente:** Examen 24 de febrero de 2010, `sirc1003`, Pregunta 1 (8 puntos), a.

Enunciado (literal)

> Dados dos hosts A y B, con sus respectivas direcciones IP y MAC notadas como sigue: `IP_A`, `IP_B`, `MAC_A` y `MAC_B`. Describa los mensajes ARP intercambiados entre A y B que permiten a A determinar la dirección MAC de B. Deberá indicar al menos direcciones MAC e IP utilizadas en cada mensaje, tanto a nivel de los cabezales Ethernet y payload ARP.

Solución oficial (verbatim)

> **A→B ARP Request con:** `Src MAC: MAC_A` · `Dst MAC: ff:ff:ff:ff:ff:ff` · Payload: `Sender MAC Addr MAC_A`, `Sender IP Addr IP_A`, `Target MAC Addr 00:00:00:00:00:00`, `Target IP Addr IP_B`
> **B→A ARP Reply con:** `Src MAC MAC_B` · `Dst MAC MAC_A` · Payload: `Sender MAC Addr MAC_B`, `Sender IP Addr IP_B`, `Target MAC Addr MAC_A`, `Target IP Addr IP_A`

Mi resolución, con la tabla completa

#	Sentido	MAC origen	MAC destino	IP origen	IP destino
1	A → todos (request)	MAC_A (A la pone)	ff:ff:ff:ff:ff:ff (broadcast)	IP_A (la de A)	IP_B (la que busca)
2	B → A (reply)	MAC_B (B la pone)	MAC_A (unicast a A)	IP_B (la de B)	IP_A (la de A)

```
En el request, Target MAC = 00:00:00:00:00:00   ← porque A todavía no la sabe
En el reply,  Target MAC = MAC_A                ← porque A se la puso en el request
```

El punto conceptual que la cátedra quiere (es la parte b del mismo ejercicio): **ARP es "independiente" de IP**, vive en el `Ethertype 0x0806` y no en el `0x0800`. Por eso IPv6 no lo usa y sí usa **Neighbor Discovery**, que está **dentro** de ICMPv6 sobre IPv6: "*IPv6 es una implementación más limpia que IPv4*". El mismo punto se vuelve a preguntar en el `2010-12 P5`.

3.3 Examen `2010-03`, Problema 1 a — tablas de R1, R2 y los hosts

> **Fuente:** Examen 24 de febrero de 2010, `sirc1003`, Problema 1 (30 pts), a. Topología: tres subredes /24 (`192.168.0.0/24`, `192.168.1.0/24`, `192.168.2.0/24`) con 100 hosts cada una, interconectadas por R1 y R2, enrutamiento estático.

Enunciado (literal)

> **a) i.** Indique las tablas de enrutamiento de los routers R1 y R2 de forma que permita conectividad extremo a extremo entre todos los equipos de la red. **ii.** Indique las tablas de enrutamiento para los hosts de cada subred.

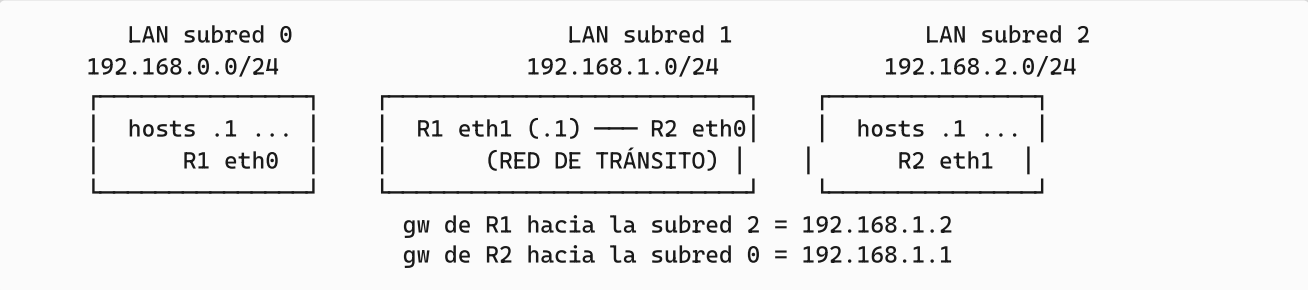
Solución oficial (verbatim)

```
Router R1                          Router R2
Network        Gateway  Interface  Network        Gateway   Interface
192.168.0.0/24      -    eth0      192.168.2.0/24      -    eth1
192.168.1.0/24      -    eth1      192.168.1.0/24      -    eth0
192.168.2.0/24  192.168.1.2  eth1   192.168.0.0/24  192.168.1.1  eth0

Hosts subred 1                      Hosts subred 2                Hosts subred 3
192.168.0.0/24      -    eth0      192.168.1.0/24      -   eth0  192.168.2.0/24   -  eth0
0.0.0.0/0     192.168.0.1 eth0     0.0.0.0/0     192.168.1.1 eth0 0.0.0.0/0  192.168.2.1 eth
```

Mi resolución: leer la tabla, que es lo que no es obvio

La tabla oficial parece rara ("¿R1 tiene conectada la subred 1 **y** usa `192.168.1.2` como gateway **de la subred 1**?"), pero es coherente: **la subred `192.168.1.0/24` es la red de tránsito** entre R1 y R2.



⚠️ **El error típico con esta topología** es poner `192.168.1.2` en la columna **Interface** en lugar de la de **Gateway**, o creer que por estar en la misma red el next hop no hace falta. **El next hop siempre se pone**, aunque sea local: sin esa columna, el router no sabe a qué MAC armar la trama de salida.

⚠️ **Y la última línea de R1:** `192.168.2.0/24 → 192.168.1.2 → eth1`. Como el next hop `192.168.1.2` está en una red **directamente conectada** (eth1), la distancia es **1**, no 2: los saltos se cuentan desde **este** router.

3.4 Examen `2009-12`, Problema 1 a — tablas y la secuencia completa de tramas

> **Fuente:** Examen 18 de diciembre de 2009, `sirc0912`, Problema 1 (30 pts), a. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf

Topología. Tres subredes de clase C: `192.168.111.0/24` (LAN1, con R1 en `192.168.111.1`), `192.168.108.0/24` (LAN2, con R1 en `192.168.108.1` y R2 en `192.168.108.2`) y `192.168.109.0/24` (LAN3, con R2 en `192.168.109.1`).

a-i) y a-ii) Tablas de routers y de hosts

```
Router R1                                Router R2
Network        Gateway  Interface        Network        Gateway   Interface
192.168.111.0/24    -    eth0            192.168.109.0/24    -    eth1
192.168.108.0/24    -    eth1            192.168.108.0/24    -    eth0
192.168.109.0/24  192.168.108.2  eth1     192.168.111.0/24  192.168.108.1  eth0

Hosts LAN1        Hosts LAN2           Hosts LAN3
111.0/24 → C      108.0/24 → C          109.0/24 → C
0.0.0.0/0 →       0.0.0.0/0 →          0.0.0.0/0 →
    192.168.111.1     192.168.108.1          192.168.109.1
```

a-iii) La secuencia completa para mandar un paquete de C a F

Paquete IP: **origen** `192.168.108.3` (host C, en LAN2) → **destino** `192.168.109.3` (host F, en LAN3). Tablas ARP **vacías**.

MACs de la figura: host C = `00-08-74-4C-7B-1C` · R1 eth1 = `00-00-0C-4C-7F-1B` · R2 eth0 = `00-00-0C-4C-8F-1A` · R2 eth1 = `00-00-0C-4C-8F-1B` · host F = `00-08-74-4C-7A-1D`.

#	MAC origen	MAC destino	IP origen	IP destino	Proto	Tipo
1	00-08-74-4C-7B-1C	FF-FF-FF-FF-FF-FF	192.168.108.3	–	ARP	Cons
2	00-00-0C-4C-7F-1B	00-08-74-4C-7B-1C	192.168.108.1	192.168.108.3	ARP	Resp
3	00-08-74-4C-7B-1C	00-00-0C-4C-7F-1B	192.168.108.3	192.168.109.3	IP	Data
4	00-00-0C-4C-7F-1B	FF-FF-FF-FF-FF-FF	192.168.108.1	–	ARP	Cons
5	00-00-0C-4C-8F-1A	00-00-0C-4C-7F-1B	192.168.108.2	192.168.108.1	ARP	Resp
6	00-00-0C-4C-7F-1B	00-00-0C-4C-8F-1A	192.168.108.3	192.168.109.3	IP	Data
7	00-00-0C-4C-8F-1B	FF-FF-FF-FF-FF-FF	192.168.109.1	–	ARP	Cons
8	00-08-74-4C-7A-1D	00-00-0C-4C-8F-1B	192.168.109.3	192.168.109.1	ARP	Resp
9	00-00-0C-4C-8F-1B	00-08-74-4C-7A-1D	192.168.108.3	192.168.109.3	IP	Data

Datagramas 1–2, 4–5 y 7–8 son ARP. Los 3, 6 y 9 son el paquete IP, en tres enlaces distintos.

Las tres cosas que hay que decir después de la tabla (es lo que convierte la tabla en respuesta de 10 puntos):

1. **Son 9 tramas porque hay 3 tránsitos y en cada uno hay que resolver una MAC nueva.** En cada salto cambian la MAC origen y la MAC destino; **la IP origen y la IP destino son siempre las mismas** (extremo a extremo).
2. **La primera trama va al gateway por defecto**, porque `192.168.109.3` no es de la subred de C (`108.0/24`). Es §1.4 aplicado al host.
3. **Entre las tramas 3 y 6 solo cambia la capa 2:** R1 tira el cabezal Ethernet y el `Ethertype`, lee el IP destino, busca en su tabla (`192.168.109.0/24 → 192.168.108.2`) y por eso necesita la MAC de R2

(**trama 4**). Lo mismo entre 6 y 9.

⚠️ **Errores típicos:** olvidar que **cada** router necesita su propio ARP (son 3 requests, no 1); poner la IP del router de destino en lugar de la del host; y olvidar que en la trama de ARP la columna **IP destino va vacía** en el request, porque el paquete ARP todavía no tiene destino IP.

PARTE 4 — CIERRE

📌 Resumen

1. Formato de 3 columnas: **Red destino/Máscara · Gateway (next hop) · Distancia**. **"C" = conectada**, distancia **1**, y nunca se borra.
2. Tres tipos: **conectada** (la pongo yo), **aprendida** (de un vecino) y **por defecto** (`0.0.0.0/0`). Sin default, el paquete se descarta con ICMP tipo 3.
3. `D & máscara == red` es el test de coincidencia, y gana el **prefijo más largo**, no el de menor distancia.
4. Un router **no** reescribe las IP al reenviar: solo cambia las tramas de capa 2 de su enlace de salida.
5. ARP existe porque **IP es jerárquica y lógica, MAC es plana y física**, y la trama necesita obligatoriamente una MAC destino.
6. 14 bytes de Ethernet + 28 de payload. En el **request** el `Target MAC` es `00:00:00:00:00:00`; en el **reply** es la MAC de quien preguntó.
7. **Request broadcast, reply unicast**: el que pregunta no sabe a quién preguntar; el que responde sí sabe a quién contestar.
8. La tabla ARP es un **caché con aging** (valores de implementación), el **gratuitous** se sigue usando para anuncio, failover y VRRP — y sigue siendo la fuente clásica del ***ARP spoofing***.
9. **RARP** (`RFC 903`) murió: solo daba la IP, sin máscara ni gateway, y necesitaba una entrada por host. Lo reemplazaron **BOOTP** y después **DHCP**.
10. En IPv6, **ARP es NDP** (NS/NA sobre ICMPv6, en multicast) — por eso el `2010-03 P1b` lo llama "más limpio".

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un router tiene `192.168.1.0/24 → C`, `192.168.1.128/25 → 10.0.0.2`, `0.0.0.0/0 → 10.0.0.1`. ¿Por dónde sale un paquete a `192.168.1.200`?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Por qué el ARP Request va en broadcast y el Reply en unicast?

<details>
<summary>Respuesta</summary>
</details>

3. Escribí los cuatro campos de direcciones de un ARP Request de H (`192.168.1.25`, MAC 2) que pregunta por `192.168.1.1`, y los del Reply de R1.

<details>
<summary>Respuesta</summary>
</details>

4. Un paquete va de H (`192.168.1.25`) a `219.41.31.21` a través de R1 y vuelve. ¿Cuáles son las cuatro IP y las dos MAC en cada sentido?

▶ Respuesta

5. ¿Por qué ARP no cruza un router?

▶ Respuesta

6. Un host tiene dos interfaces. ¿Puede un paquete salir por las dos a la vez?

▶ Respuesta

7. ¿Por qué RARP quedó obsoleto y qué lo reemplazó?

▶ Respuesta

8. ¿Qué cambia de todo esto en IPv6?

▶ Respuesta

🔗 Referencias

- `eirc1103` / `sirc1103` (21-feb-2011) — Problema 2 a, b.i, b.ii: tablas de R1 y H, secuencia ARP, MACs en los dos sentidos (§3.1) · d: subredes (Clase 28)
- `sirc1003` (24-feb-2010) — Pregunta 1 a: ARP entre A y B (§3.2) · Problema 1 a: tablas de R1, R2 y hosts (§3.3)
- `sirc0912` (18-dic-2009) — Problema 1 a-i-iii: tablas y la secuencia de 9 tramas (§3.4) · b: qué pasa al cambiar R2 por un switch
- `sirc0608` (5-ago-2006) — Pregunta 2: por qué ARP y cómo funciona (§1.5–1.7)
- `sirc0102` (2-mar-2001) — Problema 1: `selec_tarjeta`, host con dos interfaces (§2.1)
- `sirc0712` (1-dic-2007) — Problema 2: ARP con destino local vs. remoto; y dos hosts compartiendo una IP
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red"
- **RFC 826** — ARP · **RFC 903** — RARP 🔍 · **RFC 951** — BOOTP 🔍 · **RFC 2131/2132** — DHCP 🔍
- **RFC 1027** — Proxy ARP · **RFC 4443** — ICMPv6 (NDP) · **RFC 8200** — IPv6
- **Clase anterior:** 29 — Taller de subnetting · **Clase siguiente:** 31 — NAT, PAT y sus hacks